clear line between maintenance and extension. For that reason I like to combine these two properties in the context of agile projects and certainly in the context of projects that practice CD.

8.3. Measuring maintainability

In an agile and CD context, maintainability means more than updating code; it also means everything that goes into delivering changes to your consumers. As you saw in figure 8.3, that includes your build and deploy systems. When you're looking at maintainability you need to look at all of the properties of your codebase that contribute to easier code updates and faster deploy times.

Maintainability encompasses your entire development cycle and as a result is best measured as the aggregate of data from several different systems. The main components of maintainability are outlined here:

- ***Mean time to repair (MTTR)*** —The measure of time from when you realize something is wrong in production, the issue is triaged, and a fix is determined and deployed.
- ***Lead time*** —The measure of time between the definition of a new feature and when it gets to the consumer.
- ***Code coverage*** —The amount of code measured in LOC that is covered by a unit test.
- ***Coding standard rules*** —How well your code adheres to standards of the language you're using.
- ***How much code must be changed for features or bug fixes*** —The CLOC associated with tasks that go all the way through the development cycle.
- ***Bug rates*** —The number of bugs that are generated as new features are being delivered.

If maintainability is measured by the ability to make frequent changes, then the two most important metrics in this list are MTTR and lead time because they both measure the amount of time it takes to get changes to your consumers.